Homework 4: Revise System Requirements and Derive Acceptance Tests

Name: Alvin Ramcharran Date: 10/4/2026

Part 1: 1a Review and Revise SR-01 through SR-12 (check Slide Lab 6)

My revised SRs follow the SR-01 through SR-12 order from the Lab 6 slides. SR-05 is split into SR-05A through SR-05F in the Special Requirement section below, which brings the total to 17 SRs. Every SR is judged from the outside, using console output, error output, exit status, working file contents, or other MiniGit commands. No SR requires a specific internal folder, file, or data structure.

Initialization

SR-01 (source UR-GIT-01)

Given a folder that is not yet a MiniGit project and contains notes.txt with content ONE, when minigit init is used, MiniGit shall exit with status 0, make the folder a MiniGit project with nothing staged and no recorded checkpoints, and leave notes.txt byte-for-byte unchanged.

Check: Exit status is 0. notes.txt still reads ONE. minigit status reports notes.txt as untracked, and minigit log reports that no checkpoints exist.

SR-02 (source UR-GIT-01, UR-GIT-08, UR-GIT-09)

Given an initialized project with one checkpoint ("first checkpoint", notes.txt = ONE), and notes.txt then edited to TWO and staged, when minigit init is used again, MiniGit shall reject the command with an error message stating that the project is already initialized and a nonzero exit status. It shall leave the working notes.txt (TWO), the staged notes.txt (TWO), and the checkpoint history unchanged.

Check: Error output says the project is already initialized and the exit status is nonzero. Status and log output match before and after. notes.txt still reads TWO.

Selecting files

SR-03 (source UR-GIT-05)

Given an initialized project with no checkpoints, nothing staged, notes.txt = ONE and plan.txt = PLAN, when minigit add notes.txt is used, MiniGit shall exit with status 0, stage the exact current content of notes.txt (ONE), leave plan.txt unstaged, and leave both working files unchanged.

Check: Status reports notes.txt as staged and plan.txt as untracked. Both working files are unchanged. After notes.txt is edited to TWO, minigit diff shows ONE as BEFORE.

SR-04 (source UR-GIT-05, UR-GIT-08, UR-GIT-09)

Given an initialized project with no checkpoints, notes.txt staged as ONE, and no file named missing.txt, when minigit add missing.txt is used, MiniGit shall report an error identifying missing.txt as not found and exit with a nonzero status. It shall leave the staged notes.txt (ONE), the working notes.txt, and the checkpoint history unchanged.

Check: Error output names missing.txt and the exit status is nonzero. Status and log output match before and after. notes.txt still reads ONE.

Status

SR-05 (source UR-GIT-02)

Split into six sub-requirements, SR-05A through SR-05F, in the Special Requirement section below.

Comparing versions

SR-06 (source UR-GIT-03)

Given an initialized project with no checkpoints, notes.txt staged as ONE, and the working notes.txt then edited to TWO, when minigit diff is used, MiniGit shall exit with status 0 and show a whole-file comparison with the staged content ONE as BEFORE and the working content TWO as AFTER. It shall not change the staged version or the working version.

Check: Diff output shows BEFORE = ONE and AFTER = TWO. notes.txt still reads TWO. Status output matches before and after, and a second diff gives the same output.

SR-07 (source UR-GIT-04)

Given an initialized project whose latest checkpoint ("first checkpoint") contains notes.txt = ONE, with notes.txt then edited to TWO and staged, when minigit diff --staged is used, MiniGit shall exit with status 0 and show a whole-file comparison with the latest checkpoint content ONE as BEFORE and the staged content TWO as AFTER. It shall not change the checkpoint or the staged version.

Check: diff --staged output shows BEFORE = ONE and AFTER = TWO. Status and log output match before and after. notes.txt still reads TWO.

Recording and history

SR-08 (source UR-GIT-06)

Given an initialized project with no checkpoints, notes.txt staged as ONE, and the working notes.txt then edited to TWO without using add again, when minigit commit -m "first checkpoint" is used, MiniGit shall exit with status 0 and record a complete checkpoint with a new numbered ID, the exact message "first checkpoint", and the staged content notes.txt = ONE (not the working TWO). It shall leave the working notes.txt at TWO and clear the stage so nothing is staged after the commit.

Check: Log shows one entry with a numbered ID and the message "first checkpoint". notes.txt still reads TWO. Status reports notes.txt as modified with nothing staged. After minigit add notes.txt, diff --staged shows BEFORE = ONE.

SR-09 (source UR-GIT-07)

Given an initialized project, when minigit log is used, MiniGit shall exit with status 0 and list every checkpoint from newest to oldest, each with its numbered ID and exact message. If no checkpoints exist, MiniGit shall exit with status 0 and report that there are no checkpoints. In both cases log shall not change any working file, staged content, or checkpoint.

Check: With checkpoints "add notes" then "update notes", the "update notes" entry appears first and each entry shows its own ID and message. With no checkpoints, the output says there are none and the exit status is 0.

Failure behavior

SR-10 (source UR-GIT-05, UR-GIT-08, UR-GIT-09)

Given an initialized project with notes.txt staged as ONE and a file outside.txt in the folder that contains the project, when minigit add ../outside.txt is used, MiniGit shall report an error identifying ../outside.txt as outside the project and exit with a nonzero status. It shall leave the staged content, the checkpoint history, and outside.txt unchanged.

Check: Error output names the path as outside the project and the exit status is nonzero. Status and log output match before and after. outside.txt is byte-for-byte unchanged.

SR-11 (source UR-GIT-06, UR-GIT-08, UR-GIT-09)

Given an initialized project with no checkpoints and notes.txt staged as ONE, when minigit commit -m "" is used, MiniGit shall report an error stating that the commit message must not be empty and exit with a nonzero status. It shall record no checkpoint and leave notes.txt staged as ONE.

Check: Error output says the message must not be empty and the exit status is nonzero. Log still reports no checkpoints. Status still reports notes.txt as staged.

SR-12 (source UR-GIT-09, UR-GIT-08)

Given an initialized project with one checkpoint ("first checkpoint", notes.txt = ONE), notes.txt edited to TWO and staged, and plan.txt = PLAN untracked, when any MiniGit command is rejected or fails (for example add missing.txt, add ../outside.txt, commit -m "", or a repeated init), MiniGit shall exit with a nonzero status. It shall leave every working file, all staged content, and the checkpoint history exactly as they were, so a corrected command can still succeed afterward.

Check: After each rejected command, the files match and the status, log, and diff --staged output match what they were before. A following commit -m "second checkpoint" exits 0 and log shows two checkpoints.

Review checks

SR	Traces to UR	Observable	Checkable	Preserves named state
SR-01	UR-GIT-01	Exit status, status, log	Yes	notes.txt
SR-02	UR-GIT-01, 08, 09	Error output, exit status, status, log	Yes	notes.txt, staged notes.txt, checkpoint history
SR-03	UR-GIT-05	Exit status, status, diff	Yes	notes.txt, plan.txt (unstaged)
SR-04	UR-GIT-05, 08, 09	Error output, exit status, status, log	Yes	Staged notes.txt, notes.txt, checkpoint history
SR-05A	UR-GIT-02	Status output, exit status	Yes	plan.txt, stage, checkpoint history
SR-05B	UR-GIT-02	Status output, exit status, log	Yes	notes.txt, staged content, checkpoint history
SR-05C	UR-GIT-02	Status output, exit status, diff --staged	Yes	Working notes.txt (THREE), staged content (TWO),

SR	Traces to UR	Observable	Checkable	Preserves named state
				checkpoint history
SR-05D	UR-GIT-02	Status output, exit status	Yes	notes.txt (TWO), checkpoint history
SR-05E	UR-GIT-02	Status output, exit status, log	Yes	notes.txt stays missing, checkpoint history
SR-05F	UR-GIT-02	Status output, exit status	Yes	notes.txt, checkpoint history
SR-06	UR-GIT-03	Diff output, exit status	Yes	Staged and working notes.txt
SR-07	UR-GIT-04	Diff --staged output, exit status	Yes	Checkpoint and staged notes.txt
SR-08	UR-GIT-06	Log, status, diff --staged, exit status	Yes	Working notes.txt (TWO)
SR-09	UR-GIT-07	Log output, exit status	Yes	Working files, staged content, checkpoints
SR-10	UR-GIT-05, 08, 09	Error output, exit status, status, log	Yes	Staged content, checkpoint history, outside.txt
SR-11	UR-GIT-06, 08, 09	Error output, exit status, log, status	Yes	Staged notes.txt, checkpoint history
SR-12	UR-GIT-09, 08	Exit status, status, log, diff --staged	Yes	All working files, staged content, checkpoint history

Special Requirement: Expand SR-05 Status Coverage

The six status SRs follow the same rules. minigit status exits with status 0, it may report more than one state for a file, and it does not change any working file, staged content, or checkpoint. Each case starts from a **different Given condition**.

SR-05A Untracked (source UR-GIT-02)

Given an initialized project with no checkpoints, nothing staged, and plan.txt = PLAN in the project folder, when minigit status is used, MiniGit shall exit with status 0 and report plan.txt as untracked, not as staged, modified, or deleted. It shall leave plan.txt, the stage, and the checkpoint history unchanged.

Check: Status output lists plan.txt as untracked. plan.txt still reads PLAN. A second status run gives the same output.

SR-05B Staged (source UR-GIT-02)

Given an initialized project whose latest checkpoint contains notes.txt = ONE, where notes.txt was edited to TWO and minigit add notes.txt was used (staged = TWO, working = TWO), when minigit status is used, MiniGit shall exit with status 0 and report notes.txt as staged, not as changed after staging or modified. It shall leave notes.txt, the staged content, and the checkpoint history unchanged.

Check: Status output lists notes.txt as staged only. notes.txt still reads TWO. Log output matches before and after.

SR-05C Changed after staging (source UR-GIT-02)

Given an initialized project whose latest checkpoint contains notes.txt = ONE, with notes.txt staged as TWO and the working notes.txt then edited to THREE, when minigit status is used, MiniGit shall exit with status 0 and report notes.txt as changed after staging. It may also report the staged change. It shall leave the working notes.txt (THREE), the staged content (TWO), and the checkpoint history unchanged.

Check: Status output includes notes.txt as changed after staging. notes.txt still reads THREE. minigit diff --staged output matches before and after.

SR-05D Modified (source UR-GIT-02)

Given an initialized project whose latest checkpoint contains notes.txt = ONE, nothing staged since that checkpoint, and the working notes.txt edited to TWO, when minigit status is used, MiniGit shall exit with status 0 and report notes.txt as modified, not as staged. It shall leave notes.txt (TWO) and the checkpoint history unchanged.

Check: Status output lists notes.txt as modified and not staged. notes.txt still reads TWO.

SR-05E Deleted (source UR-GIT-02)

Given an initialized project whose latest checkpoint contains notes.txt = ONE, nothing staged, and notes.txt removed from the project folder outside of MiniGit, when minigit status is used, MiniGit shall exit with status 0 and report notes.txt as deleted. It shall not recreate notes.txt and shall leave the checkpoint history unchanged.

Check: Status output lists notes.txt as deleted. notes.txt is still missing from the folder. Log output matches before and after.

SR-05F Clean (source UR-GIT-02)

Given an initialized project whose latest checkpoint contains notes.txt = ONE, nothing staged, and the working notes.txt still ONE, when minigit status is used, MiniGit shall exit with status 0 and report notes.txt as clean, not as untracked, staged, modified, or deleted. It shall leave notes.txt and the checkpoint history unchanged.

Check: Status output reports notes.txt as clean and lists no other state for it. notes.txt still reads ONE.

What I changed from my original SRs

Merge note

My original SR-08 and SR-09 both described the same commit. SR-08 covered recording the staged content and message, and SR-09 covered committing after the working copy was edited without add. I merged them into the revised SR-08, which now also says the stage is cleared after a successful commit. That left the SR-09 slot open, so I renumbered my original SR-10, SR-11, and SR-12 to match the Lab 6 slide order and made the new general preservation rule SR-12. The total is 17 SRs.

Revised SR	Original (HW3)	Change
SR-01	SR-01	The original started from an empty folder. Now an existing notes.txt must survive init, and exit status 0 is required. I removed the "tracking structures" wording.
SR-02	SR-02	The behavior changed. A repeated init is now rejected with an error and a nonzero exit status instead of silently doing nothing.

Revised SR	Original (HW3)	Change
SR-03	SR-03	Added exit status 0 and that working files stay unchanged.
SR-04	SR-04	Added a nonzero exit status and named the checkpoint history as preserved.
SR-05A to 05F	SR-05	The original only covered the staged case. It is now six states, each with its own Given, and status is stated to change nothing.
SR-06, SR-07	SR-06, SR-07	Added exit status 0 and that diff changes neither version.
SR-08	SR-08 and SR-09	Merged into one SR (see the merge note). Commit records the staged content, keeps later working edits, and clears the stage after a successful commit.
SR-09	SR-11	Renumbered. Added the empty history case and that log changes nothing.
SR-10	SR-12	Renumbered. Added a nonzero exit status and named the preserved state.
SR-11	SR-10	Renumbered. Added a nonzero exit status and that notes.txt stays staged.
SR-12	None	New. Any rejected or failed command keeps all existing work and allows a retry.

1b. Revision Reflection

The most difficult part of my first SR submission was describing behavior when something goes wrong, especially the error message for an empty commit message (my original SR-10). I know how much of a pain a vague error message is, so I took extra care rereading that SR to make sure the error was specific. Of the four checks, preservation gave me the most trouble. In Lab 6

I marked six of my SRs with a question mark for preservation because they did not name exactly what had to stay unchanged.

The SR that changed the most was SR-02. When I revised it, I had to be more particular, and I saw that it was missing parts it needed to be a proper SR. It now rejects a repeated init with an error and a nonzero exit status, and it names what has to stay the same. I also merged my original SR-08 and SR-09. After reassessing the slides, I saw that the second commit case could be encompassed in SR-08, which was cleaner. SR-05 also grew from one staged case into six status cases.

After revising, I understand how to set up proper prior conditions for a requirement. SR-01 is a prime example. It started from an empty folder, so it could never show that existing files survive init.

Part 2: Derive Acceptance Tests

All tests are black-box. Evidence comes from stdout, stderr, exit status, working file contents, and the output of status, diff, diff --staged, and log. The first column gives the test ID and the SR it traces to.

Required AT Table Format

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
AT-01 (SR-01)	A folder that is not a MiniGit project contains notes.txt = ONE.	minigit init	The folder becomes a project. notes.txt is preserved. Nothing is staged or recorded.	Exit status, notes.txt before and after, status output, log output.	Exit 0. notes.txt is byte-for-byte ONE. Status lists notes.txt as untracked. Log reports no checkpoints.
AT-02 (SR-02)	Initialized project. Checkpoint "first checkpoint" has notes.txt = ONE. notes.txt was edited to TWO and	minigit init	The repeated init is rejected and all existing work is kept.	stderr, exit status, notes.txt, status and log output before and after.	stderr says "already initialized". Exit status is nonzero. notes.txt reads TWO. Status and log output match before and

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
	staged.				after.
AT-03 (SR-03)	Initialized project with no checkpoints and nothing staged. notes.txt = ONE, plan.txt = PLAN.	minigit add notes.txt	notes.txt is staged with ONE. plan.txt is not staged. Working files are unchanged.	Exit status, status output, both files before and after. Then edit notes.txt to TWO and capture diff output.	Exit 0. Status lists notes.txt as staged and plan.txt as untracked. Both files unchanged by add. Diff shows BEFORE = ONE, AFTER = TWO.
AT-04 (SR-04)	Initialized project with no checkpoints. notes.txt staged as ONE. missing.txt does not exist.	minigit add missing.txt	The add is rejected with an error naming missing.txt. Staged notes.txt and history are kept.	stderr, exit status, status and log output before and after, notes.txt.	stderr names missing.txt as not found. Exit status is nonzero. Status and log output match before and after. notes.txt reads ONE.
AT-05A (SR-05A)	Initialized project with no checkpoints and nothing staged. plan.txt = PLAN.	minigit status	plan.txt is reported as untracked. Nothing changes.	Exit status, stdout, plan.txt, a second status run.	Exit 0. stdout lists plan.txt as untracked and under no other state. plan.txt reads PLAN. The second run gives the same output.
AT-05B (SR-05B)	Initialized project. Checkpoint has notes.txt = ONE. notes.txt was edited to	minigit status	notes.txt is reported as staged only. Nothing changes.	Exit status, stdout, notes.txt, log output before and after.	Exit 0. stdout lists notes.txt as staged, not as changed after staging or modified. notes.txt

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
	TWO and add notes.txt was run, so staged and working are both TWO.				reads TWO. Log output matches.
AT-05C (SR-05C)	Initialized project. Checkpoint has notes.txt = ONE. notes.txt staged as TWO, then the working notes.txt was edited to THREE.	minigit status	notes.txt is reported as changed after staging. Nothing changes.	Exit status, stdout, notes.txt, diff --staged output before and after.	Exit 0. stdout lists notes.txt as changed after staging. A staged entry may also appear. notes.txt reads THREE. diff --staged output matches before and after.
AT-05D (SR-05D)	Initialized project. Checkpoint has notes.txt = ONE. Nothing staged. Working notes.txt edited to TWO.	minigit status	notes.txt is reported as modified, not staged. Nothing changes.	Exit status, stdout, notes.txt.	Exit 0. stdout lists notes.txt as modified and not as staged. notes.txt reads TWO.
AT-05E (SR-05E)	Initialized project. Checkpoint has notes.txt = ONE. Nothing staged. notes.txt was deleted from	minigit status	notes.txt is reported as deleted and is not recreated.	Exit status, stdout, folder listing, log output before and after.	Exit 0. stdout lists notes.txt as deleted. notes.txt is still missing. Log output matches.

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
	the folder with the operating system.				
AT-05F (SR-05F)	Initialized project. Checkpoint has notes.txt = ONE. Nothing staged. Working notes.txt = ONE.	minigit status	notes.txt is reported as clean. Nothing changes.	Exit status, stdout, notes.txt.	Exit 0. stdout reports notes.txt as clean and under no other state. notes.txt reads ONE.
AT-06 (SR-06)	Initialized project with no checkpoints. notes.txt staged as ONE, then the working notes.txt was edited to TWO.	minigit diff	Shows the whole file with staged ONE as BEFORE and working TWO as AFTER. Neither version changes.	Exit status, stdout, notes.txt, status output before and after, a second diff run.	Exit 0. stdout shows BEFORE = ONE and AFTER = TWO. notes.txt reads TWO. Status output matches. The second diff gives the same output.
AT-07 (SR-07)	Initialized project. Checkpoint "first checkpoint" has notes.txt = ONE. notes.txt was edited to TWO and staged.	minigit diff --staged	Shows the whole file with checkpoint ONE as BEFORE and staged TWO as AFTER. Neither version changes.	Exit status, stdout, notes.txt, status and log output before and after.	Exit 0. stdout shows BEFORE = ONE and AFTER = TWO. notes.txt reads TWO. Status and log output match.
AT-08A (SR-08)	Initialized project with no	minigit commit -m "first checkpoint"	A checkpoint is recorded with the	Exit status, log output, notes.txt.	Exit 0. Log shows one entry with a

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
	checkpoints. notes.txt staged as ONE, then the working notes.txt was edited to TWO without add.		staged ONE and the exact message. The working TWO is left alone.	Then run add notes.txt and capture diff --staged output.	numbered ID and "first checkpoint". notes.txt reads TWO. diff --staged shows BEFORE = ONE, AFTER = TWO.
AT-08B (SR-08)	Initialized project with no checkpoints. notes.txt = ONE and staged as ONE.	minigit commit -m "first checkpoint"	The commit succeeds and the stage is cleared.	Exit status, status output after the commit, log output.	Exit 0. Status reports notes.txt as clean with nothing staged. Log shows exactly one entry, "first checkpoint".
AT-09A (SR-09)	Initialized project with two checkpoints, "add notes" first and then "update notes".	minigit log	Lists every checkpoint newest first with its ID and message. Nothing changes.	Exit status, stdout, status output before and after.	Exit 0. "update notes" appears before "add notes". Each entry shows its own numbered ID and exact message. Status output matches.
AT-09B (SR-09)	Initialized project with no checkpoints.	minigit log	Log handles an empty history without an error.	Exit status, stdout, stderr.	Exit 0. stdout says there are no checkpoints. stderr is empty. No checkpoint entries are

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
					listed.
AT-10 (SR-10)	Initialized project. notes.txt staged as ONE. outside.txt exists in the folder that contains the project.	minigit add ../outside.txt	The add is rejected as outside the project. The stage, history, and outside.txt are kept.	stderr, exit status, status and log output before and after, outside.txt.	stderr names the path as outside the project. Exit status is nonzero. Status and log output match. outside.txt is byte-for-byte unchanged.
AT-11 (SR-11)	Initialized project with no checkpoints. notes.txt staged as ONE.	minigit commit -m ""	The commit is rejected. No checkpoint is recorded and notes.txt stays staged.	stderr, exit status, log output, status output before and after.	stderr says the message must not be empty. Exit status is nonzero. Log reports no checkpoints. Status output matches, with notes.txt still staged.
AT-12 (SR-12)	Initialized project. Checkpoint "first checkpoint" has notes.txt = ONE. notes.txt was edited to TWO and staged. plan.txt = PLAN is untracked.	minigit add missing.txt, then minigit add ../outside.txt, then minigit commit -m "", then minigit init, then minigit commit -m "second checkpoint"	Each rejected command keeps all work, and the corrected commit at the end succeeds.	Exit status of each command, notes.txt and plan.txt, and status, log, and diff --staged output before and after the four rejected commands. Log after the final commit.	Each of the four rejected commands exits nonzero. Files and status, log, and diff --staged output match before and after them. The final commit exits 0 and log shows "second checkpoint" above "first

AT (SR)	Given	When	Then	Expected evidence	Judgment rules
					checkpoint".

Suggested Traceability Format

Every SR has at least one AT. There are 17 SRs and 19 ATs.

AT	Linked SR
AT-01	SR-01
AT-02	SR-02
AT-03	SR-03
AT-04	SR-04
AT-05A	SR-05A
AT-05B	SR-05B
AT-05C	SR-05C
AT-05D	SR-05D
AT-05E	SR-05E
AT-05F	SR-05F
AT-06	SR-06
AT-07	SR-07
AT-08A	SR-08
AT-08B	SR-08
AT-09A	SR-09
AT-09B	SR-09
AT-10	SR-10
AT-11	SR-11

AT	Linked SR
AT-12	SR-12